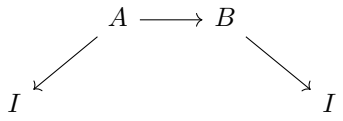
1 Polynomial Functors

Give semantics to W-Types, which should be enough for our case (hopefully). By encoding a set of operations an their arities, which we can think of as the type constructors. Where B are the operations and A the arities, we each also fibre them over a set of types. We only consider endofunctors here.



2 Given Constructors

We need the subtype to be stable under these, so we need an $F - Alg$ for the polynomial functor build from the constructors.

$$\begin{array}{c} F(\Sigma_{x:X} P(x)) \\ \downarrow{\scriptstyle \alpha} \\ \Sigma_{x:X} P(x) \end{array}$$

Therefore, we need something like $F(P)(F(x)) \implies P(x)$, from now on called a predicate being stable under F. Further if P is the smallest such predicate, so for all predicates Q stable under F we have $P(x) \implies Q(x)$ we can then $\Sigma_{x:X} P(x)$ should be initial under the Σ types, by just applying F to the obvious function. Why this is unique needs some work still.

$$\begin{array}{ccc} F(\Sigma_{x:X} P(x)) & \xrightarrow{F(f)} & F(\Sigma_{x:X} Q(x)) \\ \downarrow{\scriptstyle \alpha} & & \downarrow{\scriptstyle \delta} \\ \Sigma_{x:X} P(x) & \xrightarrow{f} & \Sigma_{x:X} Q(x) \end{array}$$

Finally, by building a prediacte which should be something like $\forall a : A \exists a' : A \;\; s.t. \;\; a = \tau(F(a'))$ we where $F(A) \to_\tau A$ is some F algebra. We should be able to build a subtype with an F-stable predicate that is isomorphic to A and therefore get initiality for all algebras.

3 Recursor for Odd and Even

In this case we can use the base recursor for $\mathbb{N}$ and build the new one from that. We do it for odd, doing it for even should be analogous, first we define a new motive:

```
Motive' : (n : Nat) -> (Odd(n)) x (Odd(n + 1)) -> Sort{u}
```

Now assume we have a motive on odd s.t.

```
Motive(one)
Motive(n) -> Motive(n + 2)
```

Then for the motive above we get

```
Motive'(zero)
Motive'(n) => Motive'(n + 1)
```

Now we have two cases either n is even and therefore n + 1 odd, then both Odd(n) and Odd(n+2) are trivially realized and we just pass along the Odd(n+1) function, so:

```
Motive'(n) = Odd(n) x Odd(n + 1) -> Sort{u}
= Odd(n + 1) -> Sort{u} = Odd(n + 1) x Odd(n + 2) -> Sort{u}
= Motive'(n + 1)
```

For the odd case, we use the fact that Motive(n) => Motive(n + 2)

```
Motive'(n) = Odd(n) x Odd(n + 1) -> Sort{u}
= Odd(n) -> Sort{u} => Odd(n + 2) -> Sort{u}
= Odd(n+1) x Odd(n + 2) -> Sort{u} = Motive'(n + 1)
```

What's left to show is that this covers all the Odd numbers and exactly those, which is covered by two lemmas

Lemma 1. $Odd(1) \wedge \forall n\,(Odd(n) \implies Odd(n+2))$

Lemma 2. $\forall n Odd(n) \implies n = 1 \vee \exists n', Odd(n') \wedge n = n' + 2$

Sketch.
We should be able to do this by using the fact that +2 is the smallest step under which Odd is closed and that Even and Odd alternate along +1. Then we first show that 1 is the smallest Odd number and then recurse down the nat expression for any Odd number, showing that the number of *succ* constructors is of the form $2n + 1$. □

4 Building Recursors

We can make this easier by using strong induction, getting rid of the requirement to handle constructor shapes in the new motive. We then always build the motive as

$$NewMotive : (i : IndType) \to (p : SubType(i)) \to motive(\langle i, p \rangle).$$

Then we build the following step function:

$$step : (i : IndType) \to ((j : IndType) \to j <_{IndOrder} i \to (pj : Subtype(j)) \to motive(\langle j, pj \rangle)) \\ \to (p : Subtype(i)) \to motive \langle i, p \rangle.$$

To then be able to apply Strong induction we need two things, we first need positivity of constructors, and we secondly need "downward closure":

1. $\forall \tau : Cons_{Subtype}$ we have that $\tau(\bar{x}) > \bar{x}$, basically stating that all inputs are subterms of $\tau(\bar{x})$.
2. $\forall \langle i, p \rangle$ we have $\tau : Cons_{Subtype}$ such that $i \in im(\tau)$.

We further need these constructors to be conservative:

$$\forall \tau : Cons_{Subtype}, im(\tau) \subseteq SubType$$

And the motive to be "stable" along the constructors

Conjecture 1. *For any subtype of an inductive type which has a set of downward closed, positive and conservative constructors there exists a recursor.*

Proof Sketch 1. *We build the function step by taking for any pair $\langle j, pj \rangle$ its downward element, which by positivity of the constructors is smaller than j. By the stability of the motive we then build the motive on $\langle j, pj \rangle$ from the motive on the downward element, giving us the function step. We then apply the strong induction principle of the inductive type to $NewMotive$ and therefore get a recursor on $SubType$.*